

TypeScript Cheat Sheet

17 tables · 241 concepts · Updated 2026-09-27 · [cheatgrid.com](#)

TypeScript is a typed superset of JavaScript that checks your code for type errors before it runs, then erases the types to leave plain JavaScript. Its type system is **structural**: if two objects have the same shape they count as the same type, whatever name either was declared with. TypeScript 6.0 turned `strict` on by default and deprecated older options such as `baseUrl` , and TypeScript 7.0, released in July 2026, is a native port of the compiler written in Go that still installs with `npm install -D typescript` . The mental model worth keeping is that types exist only while checking: the compiled JavaScript carries no type information, so data arriving at runtime is only as safe as the checks you write for it.

CONTENTS

Type Annotations & Primitive Types	1
Object Shapes (Interfaces, Aliases, Properties)	2
Type Composition (Unions, Intersections, Literals, Enums)	3
Functions (Signatures, Generics, Overloads)	4
Classes (Members, Modifiers, Inheritance)	5
Narrowing (Type Guards & Control Flow)	6
Type Operators & Advanced Types	7
Utility Types (Built-in)	8
Modules & Imports	9
TSConfig: Strictness and Type Checking	10
TSConfig: Files, Emit and Projects	12
tsc CLI Workflows	14
JSX / TSX	14
JavaScript Interop and Migration	15
Declaration Files & Augmentation	16
Decorators	17
Explicit Resource Management	18
References	18

TABLE 1

Type Annotations & Primitive Types

The starting point: how you attach a type to a value, and the built-in primitives you'll annotate with most often. Often you don't write the type at all, since inference reads it from the initializer and every type here is erased at compile time, so the real skill is knowing when an explicit annotation earns its keep. Pay special attention to the three special types near the bottom: `any` switches checking off, `unknown` is the safe top type you must narrow first, and `never` marks code that can't produce a value.

Concept	Example	Description
Type annotation	<code>let n: number = 42</code>	- Explicitly assigns a static type to a variable, parameter, or return value - erased at compile time, so it never changes runtime behavior.
Type inference	<code>const s = "hello"</code>	Compiler infers a type from an initializer or usage without an annotation <code>const</code> keeps the literal type <code>"hello"</code> , <code>let</code> widens to <code>string</code> .
string	<code>let name: string = "Ada"</code>	- Primitive text type - Always the lowercase <code>string</code> , not the rarely used built-in <code>String</code> .
number	<code>let pi: number = 3.14159</code>	Primitive numeric type (a double-precision float). JS has no <code>int</code> or <code>float</code> , so <code>number</code> covers both.

Concept	Example	Description
boolean	<code>let ok: boolean = true</code>	- Primitive true/false type - An alias for the union <code>true false</code>.
Array type	<code>let scores: number[] = [10, 20, 30]</code>	- Written <code>T[]</code>, or the equivalent generic form <code>Array<T></code> - every element must match type <code>T</code>.
Readonly array	<code>function sum(values: readonly number[]) { return values.reduce((a, b) => a + b, 0) }</code>	<code>readonly T[]</code> (shorthand for <code>ReadonlyArray<T></code>) drops mutating methods like <code>push</code> - a regular array is assignable to it, but not the reverse.
bigint	<code>const id: bigint = 123n</code>	- Primitive for arbitrary-size integers (JS <code>bigint</code>) - cannot be mixed with <code>number</code> in arithmetic.
symbol	<code>const key: unique symbol = Symbol("k")</code>	- Primitive for unique identifiers <code>unique symbol</code> supports nominal-like keys, and is allowed only on <code>const</code> and <code>readonly static</code>.
null & undefined	<code>let x: string undefined</code>	- Represent absence - require explicit handling under <code>strictNullChecks</code>.
any	<code>let data: any = JSON.parse(text)</code>	Disables type checking for the value (escape hatch) - spreads to every property you read off it, so avoid in new code.
unknown	<code>let data: unknown = JSON.parse(text)</code>	- Safer top type than <code>any</code> - accepts any value, but must be narrowed before property access or calls.
void	<code>function log(msg: string): void { console.log(msg) }</code>	Indicates a function returns no useful value . Not the same type as <code>undefined</code> .
never	<code>function fail(msg: string): never { throw new Error(msg) }</code>	Indicates no possible value : unreachable code, always-throws, or an exhausted union.
object	<code>declare function create(o: object null): void create({ prop: 0 }) // OK create(42) // Error</code>	- The non-primitive type: anything that isn't <code>number</code>, <code>string</code>, <code>boolean</code>, <code>symbol</code>, <code>bigint</code>, <code>null</code> or <code>undefined</code> - not the same as <code>Object</code> (has methods) or <code>{}</code> (anything except <code>null</code> / <code>undefined</code>); prefer <code>object</code>.

TABLE 2

Object Shapes (Interfaces, Aliases, Properties)

Most real-world typing is describing the shape of objects, and these are the tools for it. Interfaces and type aliases both name a shape, layered with optional and readonly properties, index signatures for open-ended keys, and call or construct signatures for objects that are also callable. The excess property check is the quirk worth remembering: object literals get scrutinized for stray keys in ways stored variables don't.

Pattern	Example	Description
Interface	<code>interface User { id: string; name: string }</code>	- Named object shape - supports extension and declaration merging.
Type alias	<code>type User = { id: string; name: string }</code>	Gives a name to any type expression : primitives, unions, tuples, objects.
Interface vs. type alias	<code>interface Window { title: string } interface Window { ts: string } // merges into one Window type Config = { title: string } type Config = { ts: string } // Error</code>	- A type alias can't be re-opened to add members - an interface always is.
Object type literal	<code>const u: { id: string; name: string } = { id: "1", name: "Ada" }</code>	Inline property names and types without naming the type.
Optional property	<code>type User = { name: string; email?: string }</code>	Marks a property as possibly absent , adding <code>undefined</code> to its type.

Pattern	Example	Description
Readonly property	<code>type User = { readonly id: string; name: string }</code>	Prevents assignment after initialization at the type-checking level.
Extending interfaces	<code>interface Admin extends User { role: "admin" }</code>	Builds larger shapes from smaller ones via <code>extends</code> .
Generic object type	<code>interface Box<Type> { contents: Type } let box: Box<string> = { contents: "hi" }</code>	Names a shape once and reuses it for any contained type via a type parameter.
Index signature	<code>type Headers = { [name: string]: string }</code>	Types objects whose keys aren't known in advance . Declared properties must match the value type.
Call signature	<code>type Fn = { (x: number): string }</code>	Describes a callable object that may also carry properties.
Construct signature	<code>type Ctor = { new (s: string): Date }</code>	Describes a new -able constructor type .
Excess property checks	<code>const u: User = { id: "1", name: "Ada", extra: 1 }</code>	Object literals checked for unknown properties when directly assigned to a type.
Structural typing	<code>interface Pet { name: string } class Dog { name = "Rex"; owner = "Ann" } let pet: Pet = new Dog(); // ok, shape matches</code>	TypeScript compares types by matching members , not by name or declared intent.

TABLE 3

Type Composition (Unions, Intersections, Literals, Enums)

This is where types start combining into something more expressive than a single shape. Unions say "one of these", intersections say "all of these at once", and literal types pin a value down to an exact string or number. Tuples, `as const`, and the two flavors of enum round it out, with the notes flagging which constructs emit runtime code (string and numeric enums) versus which vanish at compile time.

Type	Example	Description
Union type	<code>type ID = string number</code>	Value can be one of multiple types.
Intersection type	<code>type WithId = { id: string } & { createdAt: Date }</code>	Value must satisfy all combined types simultaneously.
Const assertion (<code>as const</code>)	<code>const roles = ["admin", "user"] as const</code>	Infers the narrowest literal and readonly types for an expression.
String literal type	<code>type Role = "admin" "user"</code>	Restricts values to specific string constants .
Numeric literal type	<code>type Dice = 1 2 3 4 5 6</code>	Restricts values to specific numeric constants .
Literal inference (widening)	<code>let a = "GET" const b = "GET"</code>	<code>let</code> widens a literal to its base type (<code>string</code>) because the value might change later <code>const</code> keeps the exact literal type
Tuple type	<code>type Args = [name: string, age?: number, ...tags: string[]]</code>	- Fixed-length array with positional element types - elements can be named, marked optional (<code>?</code> , only at the end) or a trailing rest element
Readonly tuple	<code>function move(pair: readonly [number, number]) {}</code>	- Tuple variant whose elements cannot be reassigned - the type <code>as const</code> infers on an array literal
Enum (string)	<code>enum Role { Admin = "admin", User = "user" }</code>	- Members have explicit string values at runtime - readable when logged, but get no reverse mapping - each member is its own type, so a matching plain string is not assignable where a literal union would accept it directly
Enum (numeric)	<code>enum Status { Pending, Done }</code>	- Named constants that emit a runtime JS object - auto-increment from 0, with a reverse mapping back to names.

Type	Example	Description
Const enum	<code>const enum Dir { Up, Down }</code>	• Inlines enum values at emit time, leaving no runtime object • avoid in public declaration files.

TABLE 4

Functions (Signatures, Generics, Overloads)

Typing functions covers everything from a plain parameter list to generics that carry types through a call. Generics and their `extends` constraints are the heart of reusable, type-safe utilities, while overloads let one implementation present several distinct call signatures. The smaller entries (optional, default, and rest parameters, plus newer touches like `const` type parameters) handle the everyday details of getting arguments and return values typed cleanly.

Pattern	Example	Description
Function type expression	<code>type Mapper = (x: number) => string</code>	• Describes a function's parameter and return types as a type alias • The parameter name is required
Void return type	<code>type VoidFunc = () => void</code> <code>const f: VoidFunc = () => true</code>	• A function type returning <code>void</code> still accepts an implementation that returns a value • the returned value is ignored, never an error
Generic function	<code>function first<T>(xs: T[]): T { return xs[0] }</code>	Parameterizes types with <T> for reusable, type-safe logic.
Generic constraint	<code>function len<T extends { length: number }>(x: T) { return x.length }</code>	Restricts a type parameter with <code>extends</code> so required members can be accessed.
Explicit type argument	<code>function combine<T>(a: T[], b: T[]): T[] { return a.concat(b) }</code> <code>combine<string number>([1, 2, 3], ["hello"])</code>	Overrides the type TypeScript would infer when arguments alone do not pin down a single type.
Function overloads	<code>function parse(x: string): number;</code> <code>
</code> <code>function parse(x: number): string;</code> <code>
</code> <code>function parse(x: string number) { return typeof x === "string" ? +x : String(x) }</code>	• Multiple call signatures with one implementation below them • the implementation signature is not callable from outside.
Optional parameter	<code>function greet(name?: string) {}</code>	• Parameter may be omitted • typed as possibly <code>undefined</code> inside the function.
Default parameter	<code>function inc(n: number = 1) { return n + 1 }</code>	Supplies a default value when the argument is missing or undefined , so the parameter is not <code>undefined</code> inside.
Rest parameter	<code>function sum(...ns: number[]) { return ns.reduce((a,b)=>a+b, 0) }</code>	Collects remaining arguments into a typed array . Must come last and be annotated as an array or tuple.
Parameter destructuring	<code>function sum({a, b, c}: {a: number, b: number, c: number}) { return a + b + c }</code>	Unpacks an object argument into local variables , with the type annotation placed after the destructuring pattern.
Const type parameter	<code>function identity<const T>(val: T): T { return identity(["a","b"]) }</code> // T inferred as readonly ["a","b"], not string[]	Infers the narrowest literal type from a call-site argument (TS 5.0+). Only affects literals written inside the call.
Default type parameter	<code>type ApiResult<T = unknown> = { data: T }</code>	Provides a fallback when type arguments are not supplied or inferred.
this parameter	<code>function f(this: Date) { return this.getTime() }</code>	Fake first parameter to type <code>this</code> (erased in output, never passed as an argument).

TABLE 5

Classes (Members, Modifiers, Inheritance)

TypeScript classes are JavaScript classes with a type layer on top: the same fields, methods, and inheritance, plus visibility modifiers, `readonly`, and `implements` / `abstract` for enforcing contracts. Parameter properties save you a step by declaring and assigning a field straight from the constructor signature, and the `override` keyword (with `noImplicitOverride`) makes inheritance refactors safe. Watch the erasure boundary: `private` and `readonly` are compile-time checks that vanish from the emitted JavaScript, whereas JS `#` fields stay private at runtime.

Concept	Example	Description
Class declaration	<pre>class Point { x = 0; move(n: number) { this.x += n } }</pre>	• Defines fields/methods with type annotations • emits a JS class.
Parameter properties	<pre>class User { constructor(public readonly id: string) {} }</pre>	Shorthand that declares and initializes a member from a constructor parameter.
public / private / protected	<pre>class A { private secret = 1; protected n = 2; public id = "x" }</pre>	• Controls member visibility in type checking • erased at compile time, so <code>in</code> or bracket access can still reach a <code>private</code> member at runtime.
readonly fields	<pre>class A { readonly id = "x" }</pre>	• Blocks assignment outside the constructor at type-check time • no runtime effect.
ECMAScript private fields (#x)	<pre>class Dog { #barkAmount = 0 }</pre>	Stays private after compilation , unlike TypeScript's <code>private</code> , giving hard runtime privacy instead of a compile-time check only.
Definite assignment assertion	<pre>class Greeter { name!: string }</pre>	• With <code>strictPropertyInitialization</code>, uninitialized fields error unless asserted with <code>!</code> • tells the checker the field is set some other way.
implements	<pre>class Repo implements Iterable<string> { [Symbol.iterator]() { return [][Symbol.iterator]() } }</pre>	Checks a class satisfies an interface at compile time.
extends (inheritance)	<pre>class B extends A {}</pre>	Subclassing with <code>super</code> calls and overriding.
abstract class	<pre>abstract class Base { abstract run(): void }</pre>	• Cannot be instantiated directly • may declare abstract members subclasses must implement.
Method override	<pre>class B extends A { override toString() { return "B" } }</pre>	• Replaces a base member • <code>override</code> keyword enforces the base member exists.
noImplicitOverride	<pre>// tsconfig.json { "compilerOptions": { "noImplicitOverride": true } }</pre>	Requires <code>override</code> keyword on all overriding members for safer refactors .
Accessors (get/set)	<pre>class A { get x() { return this._x } set x(v: number) { this._x = v } }</pre>	Declares typed property accessors .
Static initialization block	<pre>class C { static count = 0; static { C.count = 1 } }</pre>	Runs complex static setup code once during class initialization, with access to private members (TS 4.4+).
Generic classes	<pre>class Box<Type> { contents: Type; constructor(value: Type) { this.contents = value } }</pre>	Type parameters are inferred from the constructor call when the class is instantiated with <code>new</code> .
this type / this-based type guards	<pre>class FileSystemObject { isFile(): this is FileRep { return this instanceof FileRep } }</pre>	• The special <code>this</code> type refers to the current class, letting subclass methods return their own type • <code>this is Type</code> narrows the target in an <code>if</code> branch.
Mixins (class expression pattern)	<pre>function Scalable<T extends Constructor>(Base: T) { return class extends Base { scale = 1 } }</pre>	Combines behavior from multiple sources by returning a class expression that extends a generic base .

TABLE 6

Narrowing (Type Guards & Control Flow)

Narrowing is how a broad union type collapses to a specific one inside a branch, as the compiler watches control flow and tracks what a value can still be. The everyday guards reuse plain JavaScript checks (`typeof` , `instanceof` , `in` , truthiness, equality, assignment), discriminated unions tag each variant with a shared literal field, and type predicates, assertion functions and the exhaustiveness pattern cover what the built-in checks can't. Narrowing has real limits: it doesn't cross into a callback, and a separate lookup on the same value doesn't inherit an earlier check.

Technique	Example	Description
typeof narrowing	<code>if (typeof x === "string") x.toUpperCase()</code>	Narrows unions using JS <code>typeof</code> checks. Beware that <code>typeof null</code> is "object" .
instanceof narrowing	<code>if (e instanceof Error) console.error(e.message)</code>	Narrows via prototype chain checks. Needs a runtime value, so interfaces don't work.
in-operator narrowing	<code>if ("id" in obj) obj.id</code>	Narrows based on property existence. Optional properties stay in both branches.
Truthiness narrowing	<code>if (s) s.trim()</code>	• Narrows based on falsy/truthy JavaScript semantics • Also drops <code>" "</code> , <code>0</code> , and <code>NaN</code> .
Equality narrowing	<code>if (x === null) return</code>	Narrows unions with <code>===</code> / <code>!==</code> comparisons. Loose <code>!=</code> <code>null</code> removes <code>null</code> and <code>undefined</code> at once.
Assignment narrowing	<code>let x: string number = 10;
x = "hi"; // x: string</code>	Narrows to the assigned value's type , checked against the variable's declared type.
Discriminated union	<code>type Shape = { kind: "circle"; r: number } { kind: "square"; s: number }</code>	• Uses a shared literal field (<code>kind</code>) to narrow via <code>switch</code> / <code>if</code> checks • A <code>string</code> field won't narrow
Type predicate	<code>function isError(x: unknown): x is Error { return x instanceof Error }</code>	• User-defined guard that narrows a value to a specific type • Unchecked, so the compiler trusts the claim
Inferred type predicate	<code>const isString = (x: unknown) => typeof x === "string"</code>	TS 5.5+ infers <code>x is string</code> when there's no explicit return type and a single <code>return</code> refines the parameter.
Assertion function (<code>asserts</code>)	<code>function assertNonNull(x: unknown): asserts x { if (x == null) throw new Error() }</code>	Narrows by asserting a condition holds when the function returns normally.
Control flow analysis	<code>function padLeft(padding: number string, input: string) {
if (typeof padding === "number") return " ".repeat(padding) + input;
return padding + input; // padding: string
}</code>	Tracks reachability through branches so a value's type can differ at each point in the function, not only inside the guarded block.
Exhaustiveness check	<code>const _exhaustive: never = value</code>	Forces the compiler to prove all union cases are handled. Errors at build time if a case is missed.
Map <code>has()</code>/<code>get()</code> narrowing	<code>const m = new Map<string, number>();
if (m.has("k")) {
const v = m.get("k"); // still number undefined
}</code>	• <code>has()</code> doesn't narrow a later, separate <code>get()</code> on the same key • Store the <code>get()</code> result once and narrow that instead
Narrowing in callbacks	<code>function fn(obj: { name: string number }) {
if (typeof obj.name === "string") {
setTimeout(() => obj.name.toLowerCase()); // Error
}
}</code>	• A guard doesn't carry into a callback since the value could change before it runs • capture it in a local <code>const</code> first

TABLE 7

Type Operators & Advanced Types

This is type-level programming: operators that compute new types from existing ones rather than just declaring them. `keyof`, `typeof`, and indexed access read structure out of other types; mapped types, conditional types, and `infer` transform and branch on it; template literal types build string types programmatically. It's the most powerful corner of TypeScript and the steepest, but it's what lets a library's types stay perfectly in sync with its data. The `satisfies` operator is a gentler standout, validating a value against a type without throwing away its narrower inferred shape.

Operator	Example	Description
Type assertion (<code>as</code>)	<code>const input = document.getElementById("id") as HTMLInputElement</code>	• Overrides TypeScript's inferred type • no runtime effect. Use when you know more than the compiler.
Non-null assertion (<code>!</code>)	<code>user!.id</code>	Asserts a value is not <code>null</code> / <code>undefined</code> for type checking (erased at runtime).
Double assertion	<code>const x = value as unknown as TargetType</code>	• Bypasses the "types don't sufficiently overlap" check via the <code>unknown</code> hop • still no runtime conversion. Use sparingly.
typeof (type context)	<code>const cfg = { mode: "dev" as const }; type Cfg = typeof cfg</code>	• Extracts a type from a runtime value's shape • Legal only on identifiers and their properties
keyof	<code>type Keys = keyof User</code>	• Produces a union of property keys from an object type • a string index signature yields <code>string number</code>, not literal keys.
Indexed access type	<code>type UserId = User["id"]</code>	Looks up a property type from another type via <code>T[K]</code> . The index is itself a type.
satisfies	<code>const routes = { home: "/" } satisfies Record<string, string></code>	Checks an expression matches a type without widening the expression's own inferred type.
Mapped type	<code>type RO<T> = { readonly [K in keyof T]: T[K] }</code>	Creates a new object type by iterating over <code>keyof T</code> and applying the same transformation to each property.
Mapping modifiers (<code>-readonly</code> , <code>-?</code>)	<code>type CreateMutable<Type> = { -readonly [Property in keyof Type]: Type[Property] }; type Concrete<Type> = { [Property in keyof Type]-?: Type[Property] };</code>	• <code>-</code> (or <code>+</code>, the default) toggles a modifier while mapping • <code>-readonly</code> strips read-only, <code>-?</code> strips optionality
Key remapping (<code>as</code>)	<code>type Getters<T> = { [K in keyof T as `\${Capitalize<string & K>}`]: () => T[K] }</code>	• Renames or filters keys while mapping with the <code>as</code> clause • producing <code>never</code> drops the key.
Conditional type	<code>type IsString<T> = T extends string ? true : false</code>	Chooses a type based on an <code>extends</code> assignability test.
infer	<code>type Elem<T> = T extends (infer U)[] ? U : T</code>	• Introduces an inferred type variable inside a conditional type • In scope in the true branch only
infer with an extends constraint	<code>type ReturnType2<T> = T extends (...args: any[]) => (infer R extends string) ? R : never</code>	Constrains an <code>infer</code> variable inline, avoiding a second nested conditional to re-check it (TS 4.7+).
Distributive conditional type	<code>type ToArray<T> = T extends any ? T[] : never</code>	• Distributes over unions when the checked type is a naked type parameter • wrap both sides (<code>[T] extends [U]</code>) to turn it off.
Template literal types	<code>type EventName = `on\${Capitalize<string>}`</code>	• Builds string types via template literal syntax over string unions • Unions in several slots cross multiply
unique symbol	<code>const sym1: unique symbol = Symbol();</code>	• A <code>symbol</code> subtype tied to one specific declaration • only legal on <code>const</code> and <code>readonly static</code> members; no two <code>unique symbol</code> types are ever comparable

Operator	Example	Description
Variance annotations (<code>in</code> / <code>out</code>)	<pre>type ReadRef<out T> = { readonly value: T } type Writer<in T> = { write(x: T): void }</pre>	<code>out</code> = covariant (used in output position only) <code>in</code> = contravariant (input only); checked, not a hint: a wrong annotation is an error. Improves checker accuracy and speed (TS 4.7+).
Recursive types	<pre>type JSON = string number boolean null JSON[] { [k: string]: JSON }</pre>	A type alias that references itself - supported since TS 3.7 via deferred resolution at the alias's top level.

TABLE 8

Utility Types (Built-in)

These ship with TypeScript and save you from hand-writing the same transformations over and over. `Partial`, `Required`, `Readonly`, `Pick`, and `Omit` reshape object types; `Record` builds dictionaries; `Exclude`, `Extract`, and `NonNullable` filter unions; and `ReturnType`, `Parameters`, and `Awaited` extract pieces out of functions and promises. Two of them bite: `Partial` and `Readonly` only go one level deep, and `Omit` never checks your key against `keyof T`, so a typo silently removes nothing.

Type	Example	Description
Partial<T>	<pre>type Patch = Partial<User></pre>	Makes all properties of T optional (top level only).
Required<T>	<pre>type StrictUser = Required<User></pre>	Makes all properties of T required.
Readonly<T>	<pre>type Frozen = Readonly<User></pre>	Makes all properties of T readonly at compile time (top level only, nothing is frozen at runtime).
Pick<T, K>	<pre>type UserRef = Pick<User, "id" "name"></pre>	Selects a subset of properties by key.
Omit<T, K>	<pre>type PublicUser = Omit<User, "email"></pre>	Removes properties by key - Keys aren't checked against <code>keyof T</code>, so a typo silently removes nothing
Record<K, T>	<pre>type ById = Record<string, User></pre>	Maps keys K to value type T .
Exclude<T, U>	<pre>type NonString = Exclude<string number, string></pre>	Removes from union T the members assignable to <code>U</code> .
Extract<T, U>	<pre>type OnlyString = Extract<string number, string></pre>	Keeps from union T only the members assignable to <code>U</code> .
NonNullable<T>	<pre>type NN = NonNullable<string null undefined></pre>	Removes null and undefined from a type.
ReturnType<T>	<pre>type R = ReturnType<() => Promise<number>></pre>	Extracts a function type's return type.
Parameters<T>	<pre>type P = Parameters<(x: string, y: number) => void></pre>	Extracts a function type's parameter tuple.
Awaited<T>	<pre>type V = Awaited<Promise<string>></pre>	Recursively unwraps the resolved value type of a promise-like.
ConstructorParameters<T>	<pre>class C { constructor(a: string, b: number) {} } type P = ConstructorParameters<typeof C> // [string, number]</pre>	Extracts a constructor's parameter types as a tuple.
InstanceType<T>	<pre>type I = InstanceType<typeof User></pre>	Extracts the instance type of a constructor function type.
NoInfer<T>	<pre>function createStore<T>(init: T, fallback: NoInfer<T>): T { return init ?? fallback }</pre>	Blocks type inference from the wrapped position, forcing <code>T</code> to be inferred from other arguments (TS 5.4+).
ThisType<T>	<pre>const mixin: ThisType<{ x: number }> = { getX() { return this.x } }</pre>	Specifies the this type for methods in an object literal (requires <code>noImplicitThis</code>).

Type	Example	Description
Uppercase<S> / Lowercase<S>	<code>type U = Uppercase<"hello"> // "HELLO"</code>	Intrinsic string types that transform string literal types to upper/lower case (TS 4.1+).
Capitalize<S> / Uncapitalize<S>	<code>type C = Capitalize<"world"> // "World"</code>	Intrinsic string types that capitalize or uncapitalize the first character of a string literal type (TS 4.1+).
ThisParameterType<T>	<code>type T = ThisParameterType<(this: Date) => void> // Date</code>	Extracts the <code>this</code> parameter type from a function type.
OmitThisParameter<T>	<code>type F = OmitThisParameter<(this: Date, x: string) => void></code>	Removes the <code>this</code> parameter from a function type.

TABLE 9

Modules & Imports

How code is split across files and how TypeScript resolves and emits those boundaries: standard ESM import/export is fully type-checked, and the type-only variants (`import type` / `export type`) mark symbols that exist purely for the type system and are erased from the output. The `tsconfig` cluster here (`moduleResolution` , `module` , `esModuleInterop` , `paths`) reconciles TypeScript with the realities of Node.js and bundlers, and is a frequent source of confusing import errors. Watch the emit boundary: `paths` and `allowSyntheticDefaultImports` change type checking only, so code that compiles cleanly can still fail at runtime.

Pattern	Example	Description
ES module import/export	<code>import { readFile } from "node:fs/promises"</code>	• Standard ECMAScript module syntax, fully type-checked by TS • Any file with a top-level <code>import</code> or <code>export</code> is a module • without one it is a script in the global scope
Type-only import (<code>import type</code>)	<code>import type { User } from "./types"</code>	Imports a symbol only for the type system , guaranteed to be erased from the emitted JavaScript.
Type-only export (<code>export type</code>)	<code>export type { User } from "./types"</code>	Re-exports types without a runtime export.
Import attributes	<code>import data from "data.json" with { type: "json" }</code>	• Instructs the runtime how to load a module (e.g., JSON) • Contents are not checked by TS; they pass through to the host. Supersedes the deprecated <code>assert</code> syntax; supported from TS 5.3.
verbatimModuleSyntax	<code>// tsconfig.json
 { "compilerOptions": { "verbatimModuleSyntax": true } }</code>	Leaves any import/export without a <code>type</code> modifier in the output verbatim and drops anything marked <code>type</code> , so what you write is what you get .
ESM/CJS interop	<code>import pkg from "cjs-only"</code>	• Covers how TS models default/namespace imports across ESM and CommonJS boundaries • Node gives every CJS module a synthetic default export of its whole <code>module.exports</code> .
esModuleInterop	<code>// tsconfig.json
 { "compilerOptions": { "esModuleInterop": true } }</code>	• Adjusts emit and checking for default/namespace imports to match Node.js/bundler behavior • Adds <code>__importDefault</code> / <code>__importStar</code> helpers and implies <code>allowSyntheticDefaultImports</code> • since TypeScript 6.0 it can no longer be set to <code>false</code>
allowSyntheticDefaultImports	<code>// tsconfig.json
 { "compilerOptions": { "allowSyntheticDefaultImports": true } }</code>	• Allows <code>default</code> imports from modules without a default export (type-checking convenience only) • since TypeScript 6.0 it can no longer be set to <code>false</code> .
moduleResolution	<code>// tsconfig.json
 { "compilerOptions": { "moduleResolution": "bundler" } }</code>	• Selects how imports are resolved: <code>node16</code> , <code>nodenext</code> , or <code>bundler</code> for modern tooling; <code>bundler</code> never requires extensions on relative imports • node10 (formerly <code>node</code>) should no longer be used, <code>classic</code> was removed in TypeScript 6.0

Pattern	Example	Description
module	<pre>// tsconfig.json
 { "compilerOptions": { "module": "nodenext" } }</pre>	- Selects the emitted module format and influences resolution behavior: <code>node16</code> / <code>node20</code> / <code>nodenext</code> track Node.js, <code>preserve</code> keeps each statement's own ESM or CJS form, <code>esnext</code> is the newest spec form default since TypeScript 6.0
baseUrl / paths	<pre>// tsconfig.json
 { "compilerOptions": { "paths": { "@/*": ["src/*"] } } }</pre>	- Adds path mapping for module specifiers, type checking only - a bundler or <code>tsc-alias</code> still has to resolve it at runtime. <code>baseUrl</code> is deprecated since TypeScript 6.0 and no longer a lookup root, so fold its prefix into each <code>paths</code> entry
Namespaces	<pre>namespace Validation { export interface StringValidator { isAcceptable(s: string): boolean; } }</pre>	- Groups related types and values under one qualified name (<code>Validation.StringValidator</code>), TypeScript's own pre-ESM module system - The legacy <code>module Foo { ... }</code> spelling is a hard deprecation since TypeScript 6.0; use <code>namespace</code>
Subpath imports (#/)	<pre>// package.json
 { "imports": { "#/*": ".dist/*" } }</pre>	- Resolves import specifiers starting with <code>#</code> through the package's own <code>imports</code> field, under <code>node16</code> / <code>nodenext</code> / <code>bundler</code> TypeScript 6.0 added support for the bare <code>#/</code> prefix (previously an extra segment like <code>#root/</code> was required)
import defer	<pre>import defer * as feature from "./some-feature.js";</pre>	- Loads a module without running it until one of its exports is first accessed, namespace imports only - Added in TypeScript 5.9; only works under <code>--module preserve</code> or <code>esnext</code>, not downleveled
allowImportingTsExtensions	<pre>import * as foo from "./foo.ts";</pre>	- Lets a relative import keep a <code>.ts</code> / <code>.mts</code> / <code>.tsx</code> extension in source - only allowed with <code>noEmit</code> or <code>emitDeclarationOnly</code>, since a real JS output can't resolve that path
rewriteRelativeImportExtensions	<pre>import * as foo from "./foo.ts"; // emits "./foo.js"</pre>	- Rewrites <code>.ts</code> / <code>.tsx</code> / <code>.mts</code> / <code>.cts</code> in relative import paths to their JavaScript equivalent in emitted output - added in TypeScript 5.7, <code>paths</code> / <code>baseUrl</code> mappings are not rewritten

TABLE 10

TSConfig: Strictness and Type Checking

TypeScript's strict family of checks, plus the extra type-checking flags that sit outside it. Since TypeScript 6.0, `strict` defaults to `true`; every flag here can still be toggled on its own to ratchet strictness up or down.

Concept	Example	Description
strict	<pre>// tsconfig.json
 { "compilerOptions": { "strict": true } }</pre>	- Enables the whole strict family at once (<code>strictNullChecks</code>, <code>noImplicitAny</code>, <code>useUnknownInCatchVariables</code>, and others) Default since TypeScript 6.0; <code>noUncheckedIndexedAccess</code> and <code>exactOptionalPropertyTypes</code> stay opt-in
strictNullChecks	<pre>// tsconfig.json
 { "compilerOptions": { "strictNullChecks": true } }</pre>	Treats <code>null</code> / <code>undefined</code> as distinct types requiring explicit handling.
noImplicitAny	<pre>// tsconfig.json
 { "compilerOptions": { "noImplicitAny": true } }</pre>	Errors when the compiler would infer <code>any</code> due to missing type information - Explicit <code>any</code> is untouched
strictFunctionTypes	<pre>// tsconfig.json
 { "compilerOptions": { "strictFunctionTypes": true } }</pre>	- Checks function parameter types contravariantly instead of bivariantly, catching unsound function assignments - Part of the <code>strict</code> family

Concept	Example	Description
strictPropertyInitialization	<pre>// tsconfig.json
 { "compilerOptions": { "strictPropertyInitialization": true } }</pre>	Requires every class property to be assigned in the constructor or given a definite assignment assertion (<code>!</code>). Part of the <code>strict</code> family.
strictBindCallApply	<pre>// tsconfig.json
 { "compilerOptions": { "strictBindCallApply": true } }</pre>	Type-checks the arguments passed to <code>call</code> , <code>bind</code> and <code>apply</code> against the function's own signature instead of typing them as <code>any</code> . Part of the <code>strict</code> family.
noImplicitThis	<pre>// tsconfig.json
 { "compilerOptions": { "noImplicitThis": true } }</pre>	- Errors when <code>this</code> would resolve to an implicit any, such as inside a plain function used as a callback - One of the checks <code>strict</code> turns on
alwaysStrict	<pre>// tsconfig.json
 { "compilerOptions": { "alwaysStrict": true } }</pre>	Parses every file in ECMAScript strict mode and emits <code>"use strict"</code> in the compiled output.
useUnknownInCatchVariables	<pre>// tsconfig.json
 { "compilerOptions": { "useUnknownInCatchVariables": true } }</pre>	Types <code>catch (e)</code> as <code>unknown</code> instead of <code>any</code> , forcing you to narrow before use. Included in <code>strict</code> .
strictBuiltinIteratorReturn	<pre>// tsconfig.json
 { "compilerOptions": { "strictBuiltinIteratorReturn": true } }</pre>	- Types a built-in iterator's <code>TReturn</code> as <code>undefined</code> instead of <code>any</code>, so calling <code>.next()</code> past the end is caught - Part of <code>strict</code> since TypeScript 5.6.
noUncheckedIndexedAccess	<pre>// tsconfig.json
 { "compilerOptions": { "noUncheckedIndexedAccess": true } }</pre>	Adds <code>undefined</code> to every indexed read (<code>arr[i]</code> , <code>obj[key]</code>). Deliberately not in <code>strict</code> , since it fires even in bounds-checked loops.
exactOptionalPropertyTypes	<pre>// tsconfig.json
 { "compilerOptions": { "exactOptionalPropertyTypes": true } }</pre>	- Makes optional properties more exact: assigning <code>undefined</code> is rejected, distinct from the property being absent - Not part of <code>strict</code>.
noImplicitReturns	<pre>// tsconfig.json
 { "compilerOptions": { "noImplicitReturns": true } }</pre>	- Errors when a function has a code path that falls off the end without returning a value - Not part of <code>strict</code>.
noFallthroughCasesInSwitch	<pre>// tsconfig.json
 { "compilerOptions": { "noFallthroughCasesInSwitch": true } }</pre>	Errors on a non-empty <code>switch</code> case that falls through without <code>break</code> , <code>return</code> or <code>throw</code> . Not part of <code>strict</code> .
noUnusedLocals	<pre>// tsconfig.json
 { "compilerOptions": { "noUnusedLocals": true } }</pre>	Errors on local variables that are declared but never read . Not part of <code>strict</code> .
noUnusedParameters	<pre>// tsconfig.json
 { "compilerOptions": { "noUnusedParameters": true } }</pre>	- Errors on function parameters that are never used - Prefix a name with <code>_</code> to exempt it from the check
noPropertyAccessFromIndexSignature	<pre>// tsconfig.json
 { "compilerOptions": { "noPropertyAccessFromIndexSignature": true } }</pre>	Requires bracket notation (<code>obj["key"]</code>) rather than dot notation for a property that only exists via an index signature.
allowUnreachableCode	<pre>// tsconfig.json
 { "compilerOptions": { "allowUnreachableCode": false } }</pre>	<code>false</code> errors on unreachable code, <code>true</code> allows it silently, unset only warns in editors - Not part of <code>strict</code>
allowUnusedLabels	<pre>// tsconfig.json
 { "compilerOptions": { "allowUnusedLabels": false } }</pre>	<code>false</code> errors on unused labels, <code>true</code> allows them silently, unset only warns in editors - Not part of <code>strict</code>

TABLE 11

TSConfig: Files, Emit and Projects

The compiler options that decide which files a project scans, what a build actually emits, and how multi-package builds fit together; strictness flags live in TSConfig: Strictness and Type Checking. The trap worth knowing early: **target** downlevels **syntax** and never adds a runtime polyfill, so a build can pass clean and still crash on an older engine.

Option	Example	Description
include	<pre>// tsconfig.json
 { "include": ["src/**/*", "types/**/*d.ts"] }</pre>	• Default glob the compiler scans for source files, matched with <code>*</code>, <code>?</code> and <code>**/</code> wildcards • Falls back to <code>**/*</code> unless <code>files</code> is set
exclude	<pre>// tsconfig.json
 { "exclude": ["node_modules", "**/*.spec.ts"] }</pre>	• Removes matches from <code>include</code>'s scan; defaults to <code>node_modules</code>, <code>bower_components</code>, <code>jspm_packages</code> and <code>outDir</code> • An import can still pull an excluded file into the program
files	<pre>// tsconfig.json
 { "files": ["src/index.ts", "src/types.d.ts"] }</pre>	• Explicit allowlist of files for small projects that don't need a glob • every path must exist or the build errors
extends	<pre>// tsconfig.json
 { "extends": "./tsconfig.base.json" }</pre>	• Inherits compiler options from a base <code>tsconfig.json</code> (Node-style path resolution) • The inheriting file's <code>files</code>, <code>include</code> and <code>exclude</code> replace the base's rather than merge with them
target	<pre>// tsconfig.json
 { "compilerOptions": { "target": "ES2022" } }</pre>	• Sets the emitted JS language level (default: the current year's ES version, <code>es2025</code> today) and downlevels syntax only, never adding polyfills • Also changes the default <code>lib</code>; <code>es5</code> is deprecated since TypeScript 6.0
lib	<pre>// tsconfig.json
 { "compilerOptions": { "lib": ["ES2022", "DOM"] } }</pre>	Selects the built-in declaration libraries the checker loads (e.g., <code>DOM</code> , <code>ES2022</code>). Type definitions only: nothing from <code>lib</code> reaches your output.
rootDir	<pre>// tsconfig.json
 { "compilerOptions": { "rootDir": "./src" } }</pre>	• Input root TypeScript mirrors under <code>outDir</code>; never affects which files are compiled • Defaults to the folder holding <code>tsconfig.json</code> since 6.0, not an inferred common source directory
outDir	<pre>// tsconfig.json
 { "compilerOptions": { "outDir": "./dist" } }</pre>	Redirects emitted <code>.js</code> , <code>.d.ts</code> , <code>.js.map</code> and related files into one output folder, preserving the source's relative directory layout.
sourceMap	<pre>// tsconfig.json
 { "compilerOptions": { "sourceMap": true } }</pre>	Emits a <code>.js.map</code> next to each output file so debuggers show the original TypeScript instead of the compiled JS.
inlineSourceMap	<pre>// tsconfig.json
 { "compilerOptions": { "inlineSourceMap": true } }</pre>	• Embeds the source map as a comment inside the <code>.js</code> file instead of writing a separate <code>.js.map</code> • Mutually exclusive with <code>sourceMap</code>
declarationMap	<pre>// tsconfig.json
 { "compilerOptions": { "declaration": true, "declarationMap": true } }</pre>	Generates a source map for <code>.d.ts</code> files so an editor's Go to Definition lands on the original <code>.ts</code> source, not the declaration.
declaration	<pre>// tsconfig.json
 { "compilerOptions": { "declaration": true } }</pre>	Emits a <code>.d.ts</code> per input file describing the public types of each module.
emitDeclarationOnly	<pre>// tsconfig.json
 { "compilerOptions": { "declaration": true, "emitDeclarationOnly": true } }</pre>	Emits only <code>.d.ts</code> files and no <code>.js</code> , for when another tool (Babel, esbuild, SWC) does the transpiling.

Option	Example	Description
types	<pre>// tsconfig.json
 { "compilerOptions": { "types": ["node"] } }</pre>	- Restricts which <code>@types/*</code> packages join the global scope; empty by default since 6.0, previously every package visible under <code>node_modules/@types</code> - Doesn't affect an explicit <code>import</code> of a types-only package
typeRoots	<pre>// tsconfig.json
 { "compilerOptions": { "typeRoots": [". typings", ". vendor/types"] }</pre>	- Overrides where ambient <code>@types</code> packages are looked up - only packages under the listed folders are included, not every <code>@types</code> package on disk
skipLibCheck	<pre>// tsconfig.json
 { "compilerOptions": { "skipLibCheck": true } }</pre>	Skips type checking of <code>.d.ts</code> files themselves, speeding up builds - their types still apply to your code.
resolveJsonModule	<pre>// tsconfig.json
 { "compilerOptions": { "resolveJsonModule": true } }</pre>	Allows importing <code>.json</code> files with full type inference from their content.
isolatedModules	<pre>// tsconfig.json
 { "compilerOptions": { "isolatedModules": true } }</pre>	Errors on patterns that require cross-file type information, making single-file transpile tools safe (e.g., esbuild, SWC). Changes no output.
noUncheckedSideEffectImports	<pre>// tsconfig.json
 { "compilerOptions": { "noUncheckedSideEffectImports": true } }</pre>	- Errors when a side-effect-only <code>import "..."</code> resolves to no file, instead of silently ignoring the typo - True by default since TypeScript 6.0
noEmit	<pre>// tsconfig.json
 { "compilerOptions": { "noEmit": true } }</pre>	Runs the full type check without writing any output files - The standard CI check when another tool builds
incremental	<pre>// tsconfig.json
 { "compilerOptions": { "incremental": true } }</pre>	Saves project-graph info to a <code>.tsbuildinfo</code> file to speed up subsequent compilations.
composite	<pre>// tsconfig.json
 { "compilerOptions": { "composite": true } }</pre>	- Marks a project reference-able by other projects - Implies declaration, defaults incremental to true, and defaults rootDir to the config's folder
references	<pre>// tsconfig.json
 { "references": [{ "path": "../core" }] }</pre>	Declares dependencies between TS projects . Only <code>tsc -b</code> builds them; plain <code>tsc -p</code> reads their last-built <code>.d.ts</code> .
isolatedDeclarations	<pre>// tsconfig.json
 { "compilerOptions": { "isolatedDeclarations": true } }</pre>	- Requires exports to be annotated enough that other tools can generate <code>.d.ts</code> files without a type checker - Needs declaration or composite (TS 5.5+).
erasableSyntaxOnly	<pre>// tsconfig.json
 { "compilerOptions": { "erasableSyntaxOnly": true } }</pre>	Prohibits TS syntax with runtime semantics (enums, namespaces with runtime code, parameter properties, <code>import =</code>) so Node's type-stripping can just delete types (TS 5.8+).
ignoreDeprecations	<pre>// tsconfig.json
 { "compilerOptions": { "ignoreDeprecations": "6.0" } }</pre>	- Silences one version's deprecation errors while you migrate - TypeScript 7.0 removes the deprecated options regardless
outFile (legacy)	<pre>// tsconfig.json
 { "compilerOptions": { "outFile": "../dist/bundle.js" } }</pre>	- Concatenated global/AMD/SystemJS output into one file removed in TypeScript 6.0, use an external bundler (esbuild, Rollup, webpack) instead

TABLE 12

tsc CLI Workflows

The `tsc` invocations that cover almost everything you'll do from the command line. `tsc --noEmit` is the one CI pipelines lean on, since it type-checks without writing a single output file, while `--watch` keeps recompiling as you edit and `tsc --init` scaffolds a fresh config. The `-b` build mode is the one to know once a codebase grows into multiple referenced projects, since it rebuilds them in dependency order. Watch the one big trap: naming any file on the command line makes `tsc` ignore `tsconfig.json` completely.

Command	Example	Description
Compile project	<code>tsc</code>	- Compiles using the nearest <code>tsconfig.json</code>, searching the current directory then up the parent chain - Naming files instead makes <code>tsc</code> ignore that config
Type-check only	<code>tsc --noEmit</code>	Runs the full type check while writing no JavaScript, declarations, or source maps.
Watch mode	<code>tsc --watch</code>	- Recompiles on file changes continuously - It rebuilds only, it never re-runs your program
Initialize project	<code>tsc --init</code>	- Creates a <code>tsconfig.json</code> scaffold - minimal since TypeScript 5.9, replacing the older fully annotated version
Compile by project file	<code>tsc -p tsconfig.build.json</code>	- Compiles the project at an explicit config path, either a config file or a folder - Pass no input files alongside it
Build mode (project refs)	<code>tsc -b</code>	Builds a referenced project graph in dependency order, skipping projects already up to date.
Build with watch	<code>tsc -b --watch</code>	Incrementally rebuilds a project reference graph on changes.
Print final config	<code>tsc --showConfig</code>	Prints the fully resolved configuration <code>tsc</code> would use, without compiling anything.
List / explain included files	<code>tsc --listFiles</code> <code>tsc --explainFiles</code>	<code>--listFiles</code> prints every file read during compilation <code>--explainFiles</code> adds why each one was included
Extended diagnostics and trace	<code>tsc --extendedDiagnostics</code> <code>tsc --generateTrace ./trace</code>	<code>--extendedDiagnostics</code> prints detailed performance stats after a build <code>--generateTrace</code> writes an event trace and type list for the Chrome trace viewer
Control parallelism (TypeScript 7)	<code>tsc --checkers 8</code> <code>--builders 4</code> <code>tsc --singleThreaded</code>	- Tunes the native compiler's worker counts for type-checking and project-reference builds <code>--singleThreaded</code> disables parallelization entirely
Show TypeScript version	<code>tsc --version</code>	Prints the version of the compiler that is running.

TABLE 13

JSX / TSX

If you write React or any JSX-based UI, these are the knobs that make it type-check. The `.tsx` extension turns on JSX parsing, the `jsx` compiler option decides whether that syntax is transformed or preserved, and `jsxImportSource` points the automatic runtime at the right factory, so you can target Preact or another library instead of React. `JSX.IntrinsicElements` is where the set of valid built-in tags and their props lives.

Concept	Example	Description
TSX file extension	<code>// Component.tsx
 export function Button() { return <button /> }</code>	- Enables JSX parsing in a source file - The <code>jsx</code> compiler option must be set too
jsx	<code>// tsconfig.json
 { "compilerOptions": { "jsx": "react-jsx" } }</code>	Controls how JSX is transformed or preserved by the compiler.

Concept	Example	Description
JSX.IntrinsicElements	<pre>declare namespace JSX { interface IntrinsicElements { div: any } }</pre>	Declares valid JSX tags and their prop types for type checking.
JSX.Element (result type)	<pre>function Button(): JSX.Element { return <button /> }</pre>	- Default return type of a JSX expression - a black box with no attribute or children info
jsxImportSource	<pre>// tsconfig.json
{ "compilerOptions": { "jsx": "react-jsx", "jsxImportSource": "preact" } }</pre>	Sets the module for importing jsx / jsxs factories for the automatic runtime.
JSX.IntrinsicAttributes	<pre>interface IntrinsicAttributes { key?: string }</pre>	Extra properties a JSX framework adds outside a component's own props, such as <code>key</code> in React.

TABLE 14

JavaScript Interop and Migration

Type-checking plain JavaScript files without rewriting them: project-wide and per-file switches, JSDoc as a source of types, and emitting `.d.ts` files from JS. Declaration file authoring itself lives in Declaration Files & Augmentation.

Concept	Example	Description
allowJs	<pre>{ "compilerOptions": { "allowJs": true } } import { defaultCardDeck } from "./card"; // card.js</pre>	- Lets the compiler read and import <code>.js</code> / <code>.jsx</code> files alongside <code>.ts</code> files - does not type-check them by itself
checkJs	<pre>{ "compilerOptions": { "allowJs": true, "checkJs": true } }</pre>	Type-checks every included JavaScript file, the project-wide equivalent of adding <code>// @ts-check</code> to each one.
// @ts-check	<pre>// @ts-check let x = 0; // OK x = false; // Error: not assignable to number</pre>	Turns on type checking for a single <code>.js</code> file, the per-file equivalent of <code>checkJs</code> .
// @ts-nocheck	<pre>// @ts-nocheck let x = 0; x = false; // no error reported</pre>	Skips type checking for one file entirely, even when <code>checkJs</code> is on.
// @ts-expect-error	<pre>// @ts-expect-error const n: number = "not a number";</pre>	Suppresses the error on the next line, and itself errors if that line turns out to be error-free.
// @ts-ignore	<pre>// @ts-ignore const n: number = "not a number";</pre>	Suppresses whatever error appears on the next line, staying silent even if the line has no error.
JSDoc @type	<pre>/** @type {string number} */ let id;</pre>	Annotates a variable's type in a <code>.js</code> file with a JSDoc comment.
JSDoc @param and @returns	<pre>/** * @param {string} name * @returns {string} */ function greet(name) { return "Hi " + name; }</pre>	Documents a function's parameter and return types for a <code>.js</code> file.
JSDoc @typedef	<pre>/** @typedef {{ a: string, b: number }} SpecialType */ /** @param {SpecialType} value */ function special(value) { return value.a; }</pre>	Names a reusable object type for later <code>@type</code> or <code>@param</code> references.
JSDoc @import	<pre>/** @import { Dog } from "./dog.js" */ const d = new Dog();</pre>	Brings a type into scope from another file for JSDoc annotations only, with no runtime import.

Concept	Example	Description
Generating .d.ts from .js	<pre>{ "compilerOptions": { "allowJs": true, "declaration": true, "emitDeclarationOnly": true, "outDir": "types" } }</pre>	• <code>declaration</code> plus <code>emitDeclarationOnly</code> emit only <code>.d.ts</code> files, built from a JSDoc-typed JS project • needs <code>allowJs</code> so the compiler reads the <code>.js</code> source
maxNodeModuleJsDepth	<pre>{ "compilerOptions": { "allowJs": true, "maxNodeModuleJsDepth": 0 } }</pre>	• How many <code>node_modules</code> dependency levels the compiler infers types for from plain JS • defaults to 0.

TABLE 15

Declaration Files & Augmentation

Declaration files (`.d.ts`) carry types without any runtime code: they are how plain JavaScript packages describe their shapes, and how you fill gaps in third-party types. `declare global` and module augmentation add members to existing types (a `userId` on Express's `Request` , say) without forking the original package, and declaration merging, the mechanism behind it, happens silently with no compiler warning.

Pattern	Example	Description
Declaration file (.d.ts)	<pre>// index.d.ts export interface User { id: string }</pre>	Provides type information for JS at compile time with no runtime code.
@types package	<pre>npm install --save-dev @types/lodash</pre>	Installs community-maintained type definitions for an untyped npm package, named <code>@types/<package></code> .
Module .d.ts	<pre>// index.d.ts
 declare module "pkg" { export function f(): void }</pre>	Declares the shape of an importable module for untyped JS packages.
Ambient declare (const, function)	<pre>declare const VERSION: string; declare function greet(name: string): void;</pre>	• Declares a global variable or function that exists at runtime • no implementation, just its type
Global .d.ts	<pre>// globals.d.ts
 declare const VERSION: string</pre>	Declares globals accessible without imports (ambient global library style).
Wildcard module declaration	<pre>declare module "*.svg" { const src: string; export default src; }</pre>	An ambient module name with a single <code>*</code> wildcard matches any import path, useful for assets handled by a bundler loader.
declare global	<pre>declare global { interface Window { appVersion: string } }</pre>	Augments global types from within a module (requires at least one <code>import</code> or <code>export</code>).
Module augmentation	<pre>declare module "express-serve-static-core" { interface Request { userId?: string } }</pre>	Adds members to an existing module's declarations without forking the types package.
Declaration merging	<pre>interface Box { size: number } interface Box { color: string }</pre>	• Multiple declarations with the same name are merged into one combined type • happens silently, with no compiler warning, unlike a duplicate <code>type</code> alias, which errors
Triple-slash reference (types)	<pre>/// <reference types="node" /></pre>	• Declares a type package dependency for the file • in <code>.ts</code> files, set <code>types</code> in <code>tsconfig.json</code> instead.
Triple-slash reference (lib)	<pre>/// <reference lib="dom" /></pre>	Explicitly includes a built-in lib (e.g., <code>dom</code>) in the compilation without changing <code>tsconfig.json</code> .

TABLE 16

Decorators

Decorators wrap classes and their members to add or alter behavior, using the `@`-prefixed syntax you have seen in frameworks like Angular and NestJS. TypeScript 5.0 made the standardized TC39 decorators the default: each one receives the decorated value plus a context object and can return a replacement, and 5.2 added a metadata channel via `Symbol.metadata`. The crucial caveat is that these are not compatible with the older `experimentalDecorators` form, so a project picks one system for the whole compilation.

Decorator	Example	Description
Class decorator	<pre>function sealed(Base: Function, ctx: ClassDecoratorContext) { Object.seal(Base) } @sealed class C {}</pre>	• Wraps or replaces a class • receives the class and a context object • return a callable to replace the class, or nothing to keep it.
Method decorator	<pre>function log(fn: Function, ctx: ClassMethodDecoratorContext) { return function(this: any, ...args: any[]) { return fn.apply(this, args) } } class C { @log greet() {} }</pre>	• Wraps a method • the replacement function is returned • return nothing and the original method is left alone.
Field decorator	<pre>function init<T>(val: T) { return (_: undefined, _ctx: ClassFieldDecoratorContext) => () => val } class C { @init("Ada") name!: string }</pre>	• Intercepts field initialization • receives <code>undefined</code>, never the field's value • return an initializer function to override the default value.
Accessor decorator	<pre>function bound({ get, set }: ClassAccessorDecoratorTarget<any, string>, ctx: ClassAccessorDecoratorContext) {} class C { @bound accessor name = "Ada" }</pre>	• Decorates an accessor keyword field, which auto-generates a getter/setter pair over a private slot • receives that <code>get</code> / <code>set</code> pair and may return <code>{ get, set, init }</code>.
Getter/setter decorator	<pre>function configurable(fn: Function, ctx: ClassGetterDecoratorContext) {} class C { @configurable get x() { return 1 } }</pre>	• Decorates a getter or setter separately • <code>@configurable</code> on <code>get x()</code> leaves <code>set x()</code> undecorated.
Decorator metadata	<pre>function meta(_: any, ctx: DecoratorContext) { ctx.metadata.info = "x" } @meta class C {}; C[Symbol.metadata]</pre>	• TS 5.2+ adds <code>Symbol.metadata</code> to classes • decorators share one metadata object per class, accessible at runtime.
experimentalDecorators (legacy)	<pre>// tsconfig.json { "compilerOptions": { "experimentalDecorators": true } }</pre>	• Enables the pre-TC39 legacy decorator syntax used by Angular, NestJS, and older libraries • methods <code>get(target, propertyKey, descriptor)</code>, parameters can be decorated, and <code>emitDecoratorMetadata</code> works • incompatible with the standard decorators, and the flag applies to the whole compilation.
Parameter decorator (legacy)	<pre>function required(target: Object, propertyKey: string, parameterIndex: number) {} class BugReport { print(@required verbose: boolean) {} }</pre>	• Decorates a constructor or method parameter, valid only under <code>experimentalDecorators</code> • receives the target, member name and the parameter's ordinal index; its return value is ignored.
emitDecoratorMetadata (legacy)	<pre>// tsconfig.json { "compilerOptions": { "emitDecoratorMetadata": true } }</pre>	• Emits type metadata for decorated declarations under <code>experimentalDecorators</code> • readable at runtime once the <code>reflect-metadata</code> library is imported.

TABLE 17

Explicit Resource Management

Deterministic cleanup, built into the language: `try/finally` you no longer have to write by hand. Declare a resource with `using` (or `await using` for async cleanup) and its `[Symbol.dispose]` method runs automatically when the enclosing block exits, so connections and file handles close on an early `return` or a thrown error too. Resources are released in reverse order of declaration, the `Disposable` interface defines what makes a class eligible, and `DisposableStack` groups several resources to tear down together.

Feature	Example	Description
using declaration	<code>using conn = getConnection()</code>	• Calls <code>conn[Symbol.dispose]()</code> when the enclosing block exits, including on early <code>return</code>, <code>break</code>, and <code>throw</code> • <code>null</code> / <code>undefined</code> are a no-op (TS 5.2+, ECMAScript Stage 4).
await using declaration	<code>await using conn = await openConnection()</code>	• Calls and awaits <code>conn[Symbol.asyncDispose]()</code> when the enclosing block exits • falls back to <code>[Symbol.dispose]</code> if there is no async disposer • the <code>await</code> marks disposal only, so acquisition needs its own <code>await</code>.
Symbol.dispose	<code>class Conn { [Symbol.dispose]() { this.close() } }</code>	• The synchronous cleanup method called by <code>using</code> • implement to make a class disposable.
Symbol.asyncDispose	<code>class Conn { async [Symbol.asyncDispose]() { await this.close() } }</code>	The asynchronous cleanup method called by <code>await using</code> .
Disposable interface	<code>function use(r: Disposable) { using _ = r; }</code>	• Interface requiring <code>[Symbol.dispose]() : void</code> • available with <code>lib</code> set to <code>"esnext"</code> or <code>"esnext.disposable"</code>.
AsyncDisposable interface	<code>async function use(r: AsyncDisposable) { await using _ = r; }</code>	Interface requiring <code>[Symbol.asyncDispose]()</code> , returning a promise.
DisposableStack	<code>using stack = new DisposableStack(); stack.use(resource); stack.defer(() => cleanup())</code>	• Aggregates multiple resources for disposal in LIFO order • <code>use()</code> takes a disposable, <code>defer()</code> a callback, <code>adopt()</code> a value plus its disposer • <code>move()</code> transfers ownership.
AsyncDisposableStack	<code>await using stack = new AsyncDisposableStack(); stack.use(resource); stack.defer(async () => await cleanup())</code>	• The async counterpart of DisposableStack, for tracking <code>AsyncDisposable</code>s and async cleanup callbacks • <code>move()</code> transfers its registered disposers to a new stack.
SuppressedError	<code>err.error / err.suppressed</code>	• Wraps both errors when disposal throws while another error is already in flight • <code>error</code> is the later throw, <code>suppressed</code> the earlier one.

References

Official Documentation

1. Type annotation
2. Readonly array
3. symbol
4. unknown
5. never
6. object
7. Structural typing
8. Const assertion (as const)
9. Enum (string)
10. Generic function
11. Const type parameter
12. Class declaration
13. noImplicitOverride
14. Mixins (class expression pattern)
15. Inferred type predicate
16. Assertion function (asserts)
17. Non-null assertion (!)
18. typeof (type context)
19. keyof
20. Indexed access type
21. satisfies
22. Mapped type
23. Conditional type
24. infer with an extends constraint
25. Template literal types
26. Recursive types

27. Partial
29. Type-only import (import type)
31. verbatimModuleSyntax
33. esModuleInterop
35. moduleResolution
37. Namespaces
39. allowImportingTsExtensions
41. strict
43. noImplicitAny
45. strictPropertyInitialization
47. noImplicitThis
49. useUnknownInCatchVariables
51. noUncheckedIndexedAccess
53. noImplicitReturns
55. noUnusedLocals
57. noPropertyAccessFromIndexSignature
59. allowUnusedLabels
61. target
63. rootDir
65. sourceMap
67. declarationMap
69. emitDeclarationOnly
71. typeRoots
73. resolveJsonModule
75. noUncheckedSideEffectImports
77. incremental
79. references
81. erasableSyntaxOnly
83. Compile project
85. jsx
87. allowJs
89. // @ts-check
91. JSDoc @type
93. maxNodeModuleJsDepth
95. @types package
97. Ambient declare (const, function)
99. declare global
101. Decorator metadata
103. Parameter decorator (legacy)
105. Symbol.dispose
107. DisposableStack
109. object
111. rewriteRelativeImportExtensions
113. strictPropertyInitialization
115. noImplicitThis
117. Compile by project file -
<https://www.typescriptlang.org/docs/handbook/tsconfig-json.html>
119. Watch mode -
<https://www.typescriptlang.org/docs/handbook/configuring-watch.html>
120. ES module import/export
121. Import attributes
122. ESM/CJS interop
123. allowSyntheticDefaultImports
124. module
125. import defer
126. rewriteRelativeImportExtensions
127. strictNullChecks
128. strictFunctionTypes
129. strictBindCallApply
130. alwaysStrict
131. strictBuiltinIteratorReturn
132. exactOptionalPropertyTypes
133. noFallthroughCasesInSwitch
134. noUnusedParameters
135. allowUnreachableCode
136. include
137. lib
138. outDir
139. inlineSourceMap
140. declaration
141. types
142. skipLibCheck
143. isolatedModules
144. noEmit
145. composite
146. isolatedDeclarations
147. ignoreDeprecations
148. TSX file extension
149. jsxImportSource
150. checkJs
151. // @ts-expect-error
152. Generating .d.ts from .js
153. Declaration file (.d.ts)
154. Module .d.ts
155. Global .d.ts
156. Triple-slash reference (types)
157. experimentalDecorators (legacy)
158. emitDecoratorMetadata (legacy)
159. Symbol.asyncDispose
160. AsyncDisposableStack
161. Tuple type
162. strictFunctionTypes
163. strictBindCallApply
164. strictBuiltinIteratorReturn
165. NoInfer<T> - <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-4.html#the-noinfer-utility-type>

Source Code & Repositories

120. Map has()/get() narrowing
121. Narrowing in callbacks
122. Field decorator

Articles & Tutorials

123. Control parallelism (TypeScript 7)